CINEMATE- MOVIE RECOMMENDATION CHATBOT

Project developed by: Diana Dinca

Overview

CineMate was born from a passion for movies, coding, and natural language interaction. This project is a Python-based chatbot designed to interact with users and provide personalized movie recommendations based on genre preferences, mood, or specific actor requests.

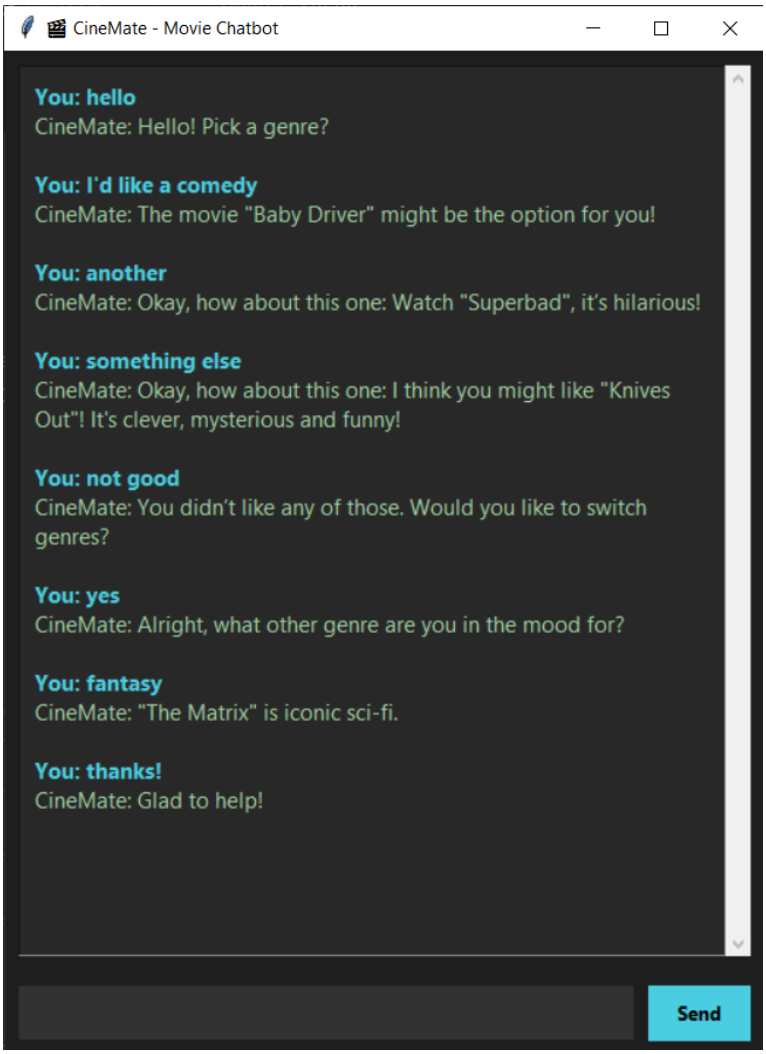
The chatbot offers intelligent, evolving conversations where the bot remembers what you like, reacts when you don’t, and even surprises you with something new. CineMate is built to feel like chatting with a well-informed movie buddy. Explore it at your pace, ask freely, and let it guide you to your next great movie night.

User Manual

CineMate is designed to be simple and intuitive to use, whether you interact with it through the terminal or its graphical user interface.

To use the chatbot effectively, simply type natural language prompts like "Recommend a comedy," "I want something romantic," or "What should I watch tonight?" The bot uses a trained neural network to interpret your intent and respond with curated movie suggestions. You can also mention specific actors (e.g., "Show me something with Emma Stone") and CineMate will offer a matching recommendation.

If you don’t like a suggestion, you can respond with phrases like "I’ve seen it," "something else," or "not good." After three such rejections in a row, the bot will automatically ask if you'd like to switch genres, helping you navigate to better-fitting options. At any time, you can ask for a surprise by saying "surprise me," and the bot will select a movie from a random genre.



Features

Genre-Based Recommendations

- Supports multiple genres: comedy, action, romance, sci-fi, horror, feel-good
- Each genre includes 15+ curated movie titles

Actor-Based Suggestions

- Recognizes specific actors like Leonardo DiCaprio, Emma Stone, Keanu Reeves, Meryl Streep and more
- Suggests personalized film picks based on actor mentions

Rejection Handling

- The bot remembers the last suggested genre
- It responds with new recommendations upon rejection
- After 3 rejections, CinMate offers to switch genres

Surprise Mode

- Users can type "surprise me" to receive a random movie from any genre

GUI Interface

- Clean, modern UI with dark mode theme
- Message log, input box, and send button
- Scrollable chat area

Technologies Used

- Python 3.10+
- PyTorch for NLP intent classification:
  - CineMate uses Natural Language Processing (NLP) to understand user input and classify their intent. This is done by training a neural network to recognize patterns in how users phrase their requests.
- NLTK to prepare the text for machine learning:
  - Tokenization – splitting sentences into individual words
  - Stemming – reducing words to their root form (e.g., “laughing” ➜ “laugh”)
  - Bag-of-Words conversion – turning words into numeric vectors for training the neural network
- Tkinter (GUI) for a clean and intuitive interface

Project Structure

The file *chat.py*

The brain of the bot. This script handles:

- Incoming user messages (from console or GUI)
- Tokenization and preprocessing
- Intent prediction using the trained model
- Logic for:
  - Genre recognition
  - Actor matching
  - Surprise mode
  - Rejection handling and context memory (last_tag, rejection_count)
- Exposes the get_bot_response() function for use in GUI

```python
print("Let's chat! (type 'quit' to stop)")
def get_bot_response(sentence):  2 usages
    global last_tag, rejection_count, ask_to_switch

    # Handle rejections
    if any(kw in sentence.lower() for kw in rejection_keywords):
        if last_tag:
            rejection_count += 1
            if rejection_count >= 3:
                ask_to_switch = True
                return "You didn't like any of those. Would you like to switch genres?"
            else:
                for intent in intents["intents"]:
                    if intent["tag"] == last_tag:
                        return f"Okay, how about this one: {random.choice(intent['responses'])}"
        else:
            return "I'm not sure which genre you want. Could you tell me again?"

    # Handle response to "switch genres?"
    if ask_to_switch:
        if "yes" in sentence.lower():
            last_tag = None
            rejection_count = 0
            ask_to_switch = False
            return "Alright, what other genre are you in the mood for?"
        elif "no" in sentence.lower():
            rejection_count = 0
            ask_to_switch = False
            for intent in intents["intents"]:
                if intent["tag"] == last_tag:
                    return f"Okay, sticking with {last_tag}. What about this: {random.choice(intent['responses'])}"
```

The file *model.py*

Defines the NeuralNet class:

- A simple feedforward neural network
- Input layer → Hidden layer → Output layer
- Used for classifying intent based on bag-of-words vectors

The file *data.json*

This contains the dataset used to train the model.

- Each entry contains:
  - tag: intent label (like “comedy” or “romance”)
  - patterns: example user inputs
  - responses: suggested movie replies
- data_updated.json includes cleaned titles and expanded movie lists

```
"intents": [
  {
    "patterns": [
      "Thanks a lot",
      "Appreciate it",
      "ok","okay","oki"
    ],
    "responses": [
      "You're welcome!",
      "Anytime! Enjoy your film!",
      "Glad to help!"
    ]
  },
  {
    "tag": "comedy",
    "patterns": [
      "I want to laugh",
      "Recommend a comedy",
      "Funny movie",
      "Make me laugh",
      "I need a comedy",
      "Something fun",
      "comedy",
      "I want a comedy movie",
      "Give me a good comedy",
      "Need a comedy"
    ],
    "responses": [
      "Try \"Meet the Parents\" or \"Big Daddy\" - guaranteed laughs!",
      "\"Step Brothers\" is a classic pick!",
      "Go for \"The Hangover\" if you're in the mood for chaos!",
      "Try Don't Look Up it's pretty popular!",
```

The file *nltk_utils.py*

Preprocessing toolkit:

- tokenize(): splits user input into words
- stem(): reduces words to their root forms
- bag_of_words(): vectorizes tokenized input into a numerical form usable by the model

The file *interface.py*

Tkinter-based graphical interface:

- Chat window (scrollable)
- Text input field
- “Send” button
- Handles interaction with the get_bot_response() from chat.py

- Offers a more user-friendly experience than console

The file *train.py*

This file is the engine behind training the Neural Network used in CineMate.

- Loads and parses the data.json intent file
- Preprocesses the texts
- Builds training data (features and labels)
- Initializes the NeuralNet defined in model.py
- Trains for a specified number of epochs (e.g. 1000)
- Displays training progress (Loss: ... every N epochs)
- Stores model parameters, vocabulary (all_words), and class labels (tags) into model.pth using torch.save()

Future Improvements

CineMate was designed with extensibility in mind, and there are many directions in which the project can evolve.

One of the most significant improvements would be to expand the training dataset. By incorporating a wider range of genres, actors, subgenres, and even international films, the chatbot could offer far more nuanced and personalized recommendations.

The current GUI could be adapted into a web-based version using Flask or React, which would make it easier to use on mobile or online platforms. Integration with services like Netflix, Amazon Prime, Disney Plus or HBO Max could turn CineMate into a smart assistant that suggests content based on real-time viewing history and preferences.

The bot could also evolve by storing user preferences, supporting session memory, offering multilingual responses, and enhancing the experience with visuals like movie posters, trailers, and rating data.